10 INHERITANCE • DETAILED SOLUTIONS

conceptual solutions

Detailed Theory, Exact C++ Rules, Memory Models & Exam Reference

Detailed Solutions — Conceptual Questions (Module 10: Inheritance)

Solution to Q1 (Default Inheritance Modes)

- **Concept**: Base-specifier defaults (`[class.access.base]`).
- **Rule**: When deriving a `class` (`class D : B`), the default inheritance mode is `private` . When deriving a `struct` (`struct D : B`), the default inheritance mode is `public` .

Solution to Q2 (Access Transformation Matrix)

- `public` base members become `public` under `public` inheritance, `protected` under `protected` inheritance, and `private` under `private` inheritance.
- `protected` base members become `protected` under `public` and `protected` inheritance, and `private` under `private` inheritance.
- `private` base members are never directly accessible inside the derived class under any inheritance mode.

Solution to Q3 (Classroom Example 1: `super` and `sub`)

- **Rule**: In `class sub : protected super` , `super::show()` (originally `public` in `super`) becomes a **`protected` member of `sub`** . Because `main()` is outside `class sub` and is not a derived class or friend of `sub` , calling `obj.show()` in `main()` violates C++ access control and fails to compile.

Solution to Q4 (Classroom Example 2: `person` and `student`)

- **Rule**: In `class student : protected person` , `person::input(int)` becomes `protected` in `student` .
  - Inside `student::display()` , the code is executing inside a member function of `student` , where `protected` members of `student` are **accessible** (`input(30)` is valid).
  - Inside `main()` , `s.input(40)` is an outside access through an object `s` of type `student` . Outside code cannot access `protected` members (`s.input(40)` is a compilation error).

Solution to Q5 (Constructor and Destructor Order)

- **Multilevel (`A → B → C`)**: Constructors run `A()` , then `B()` , then `C()` . Destructors run `~C()` , then `~B()` , then `~A()` .
- **Multiple (`class C : public A, public B`)**: Base constructors execute in declaration order (`A()` , then `B()` , then `C()`), and destructors execute in exact reverse order (`~C()` , then `~B()` , then `~A()`).

Solution to Q6 (Diamond Problem & Virtual Base Classes)

- Without `virtual` inheritance, `teaching_assistant` inherits two separate copies of `person` (one through `student` and one through `employee`), causing ambiguity when accessing `age`.
- Declaring `class student : virtual public person` and `class employee : virtual public person` merges the two `person` subobjects into a single shared virtual base subobject. The **most-derived class** (`teaching_assistant`) initializes the virtual base class `person` before any non-virtual base constructors run.